

CSC 227: Operating Systems

Project Report

Multithreaded CPU Scheduling Simulator

a. Team Members

The following table lists all team members and their student IDs. The team leader is highlighted.

#	Student Name	Student ID
1	Jood Alkhneen (Leader)	445201348
2	Lama Almubarak	445200338
3	Tala Alqahtani	445204552
4	Shatha Bin Mana	445202291
5	Rana Alsalman	445201204

b. Task Distribution Table

Tasks were distributed clearly and practically among all team members as follows:

Student Name	Tasks Performed
Jood Alkhneen (Leader)	- Designed PCB & GanttEntry classes - Priority Scheduling with starvation detection and aging algorithm - Report writing and documentation
Lama Almubarak	- Implemented Thread 1 (JobLoader) - file reading, parsing job.txt, input validation, and inserting PCBs into the job queue
Tala Alqahtani	Implemented Thread 2 (MemoryManager)

Student Name	Tasks Performed
	memory checking, process admission to ready queue, and memory synchronization
Shatha Bin Mana	- Designed the SharedQueues class - thread-safe queue operations using synchronized methods, wait() and notifyAll() for inter-thread communication
Rana Alsaman	- Implemented the Scheduler - Shortest Job First (SJF) and Round Robin (RR, q=5ms) algorithms, Gantt chart generation, and performance metrics calculation
All Members	Testing all three algorithms, verifying output correctness, taking screenshots, and final review

c. Software and Hardware Tools

Software Tools

- Programming Language: Java (JDK 24)
- IDE: Visual Studio Code with Java Extension Pack
- IDE: Apache NetBeans 25
- Operating System: Windows 11
- Version Control: Manual file management

Hardware Tools

- Processor: Intel Core i7 (or equivalent)
- RAM: 8GB or higher
- Storage: SSD

d. Multithreading and Synchronization

How Multithreading Enhances Performance

In our CPU Scheduling Simulator, multithreading plays a critical role in making the simulation more realistic and efficient. The application uses three concurrent threads, each responsible for a distinct task that mirrors how a real operating system works internally.

The three threads and their roles:

- Thread 1 (JobLoader): Reads process information from job.txt and loads PCBs into the job queue. It operates independently and terminates as soon as all processes are inserted.
- Thread 2 (MemoryManager): Continuously monitors the job queue and available memory. It admits processes to the ready queue only when sufficient memory is available, simulating the OS long-term scheduler.
- Main Thread (Scheduler): Executes the chosen scheduling algorithm (SJF, Round Robin, or Priority) by picking processes from the ready queue and simulating CPU execution.

Without multithreading, all these tasks would run sequentially Thread 1 would have to finish reading the entire file before Thread 2 could start, and the scheduler would only begin after all processes were already admitted. This would be unrealistic and inefficient. With multithreading, Thread 1, Thread 2, and the Main Thread run concurrently: Thread 1 reads and inserts processes into the job queue while Thread 2 simultaneously checks memory and admits them to the ready queue, and the scheduler begins as soon as the first process is available.

This concurrent design significantly improves throughput and responsiveness, closely simulating the behavior of a real OS where the long-term, medium-term, and short-term schedulers operate in parallel.

Synchronization Mechanisms

Synchronization is absolutely required in our application because multiple threads access and modify shared data simultaneously. Without synchronization, race conditions would occur, leading to corrupted data or program crashes.

We used two synchronization mechanisms:

- SharedQueues with synchronized methods and wait/notifyAll: The SharedQueues class manages the job queue and ready queue using plain LinkedList with synchronized methods. Every method that accesses the queues is synchronized on the same object (this). When a queue is empty, threads call wait() to sleep and release the lock. When new data arrives, notifyAll() wakes sleeping threads. This ensures only one thread accesses the queue at a time, preventing data corruption.
- memoryLock object with synchronized blocks: A dedicated Object (memoryLock) is used to protect the availableMemory variable, which is shared between Thread 2 (which decreases memory on admission) and the Main Thread (which increases it when a process finishes). Thread 2 calls wait() on memoryLock when memory is insufficient, and the Main Thread calls notifyAll() inside freeMemory() after releasing memory. This ensures memory is never over-allocated. This same mechanism also handles clean thread termination, Thread 2 uses two counters, finishedCount and admittedCount, and calls wait() until both match, guaranteeing it only exits after every admitted process has fully completed.

We deliberately chose two separate locks for two separate concerns: queue operations and memory operations. This avoids holding one lock while waiting for another, which is the classic cause of deadlock.

e. Preferred Output Format: Gantt Chart vs Table

We prefer using both the Gantt chart and the process table together, but if we had to choose one primary format, we would choose the Gantt chart. Here is our justification:

Why We Prefer the Gantt Chart

- Visual clarity: The Gantt chart provides an immediate visual representation of the process execution order and timing. It is much easier to see at a glance which process ran when, and for how long.
- Preemption visibility: For Round Robin scheduling, the Gantt chart clearly shows each time slice and preemption, which a table cannot represent clearly since a process may appear multiple times at different time intervals.
- Start and stop burst values: Our Gantt chart includes starting and stopping burst values for each execution segment (e.g., P1: time 0 to 5 ms, burst 25 to 20), making it easy to trace the exact state of each process at every point in time.

Role of the Table

The process table complements the Gantt chart by providing precise numeric data for each process including start time, finish time, waiting time, and turnaround time. These values are essential for calculating performance metrics and comparing algorithms. Together, the Gantt chart and the process table give a complete and clear picture of the scheduling behavior.

f. Screenshots

The following screenshots show the output of the program using the input file shown below:

Input File (job.txt)

1:25:4;500
2:13:3;700
3:20:3;100
4:8:1;300
5:15:2;400

Format: PID : Burst Time : Priority ; Memory Required (MB)

Screenshot 1: SJF Output: Shows the Gantt chart and process table for Shortest Job First scheduling. Processes run in order of burst time: P4 (8ms) → P2 (13ms) → P5 (15ms) → P3 (20ms) → P1 (25ms). Average waiting time: 24.20ms.

```
run:
-----
CSC 227: Multithreaded CPU Scheduling Simulator
-----

Select a CPU Scheduling Algorithm:
1. Shortest Job First (SJF)
2. Round Robin (RR, q = 5ms)
3. Priority Scheduling (Non-Preemptive) with Aging
Enter your choice (1-3): 1
[Thread 2] Started: Memory Manager. Total memory = 2048MB

[Scheduler] Starting Shortest Job First Scheduling...
[Thread 1] Started: Reading jobs from job.txt
[Thread 2] P1 admitted to Ready Queue. Available = 1548MB
[Thread 1] Added to Job Queue: P1 [burst=25, remaining=25, priority=4, memory=500MB, state=new]
[Thread 2] P2 admitted to Ready Queue. Available = 848MB
[Thread 1] Added to Job Queue: P2 [burst=13, remaining=13, priority=3, memory=700MB, state=new]
[Thread 2] P3 admitted to Ready Queue. Available = 748MB
[Thread 1] Added to Job Queue: P3 [burst=20, remaining=20, priority=3, memory=100MB, state=new]
[Scheduler] Received: P1 [burst=25, remaining=25, priority=4, memory=500MB, state=ready]
[Thread 2] P4 admitted to Ready Queue. Available = 448MB
[Scheduler] Received: P2 [burst=13, remaining=13, priority=3, memory=700MB, state=ready]
[Thread 1] Added to Job Queue: P4 [burst=8, remaining=8, priority=1, memory=300MB, state=new]
[Thread 2] P5 admitted to Ready Queue. Available = 48MB
[Scheduler] Received: P3 [burst=20, remaining=20, priority=3, memory=100MB, state=ready]
[Thread 1] Added to Job Queue: P5 [burst=15, remaining=15, priority=2, memory=400MB, state=new]
[Scheduler] Received: P4 [burst=8, remaining=8, priority=1, memory=300MB, state=ready]
[Scheduler] Received: P5 [burst=15, remaining=15, priority=2, memory=400MB, state=ready]
[Thread 1] Finished: All jobs inserted into Job Queue.
[Thread 2] No more jobs from Thread 1.
[MEM-FREE] P4 completed. Freed 300MB. Available = 348MB
[MEM-FREE] P2 completed. Freed 700MB. Available = 1048MB
[MEM-FREE] P5 completed. Freed 400MB. Available = 1448MB
[MEM-FREE] P3 completed. Freed 100MB. Available = 1548MB
[MEM-FREE] P1 completed. Freed 500MB. Available = 2048MB
-----
GANTT CHART
-----
[Thread 2] Finished: All processes admitted and completed.
| P4 | P2 | P5 | P3 | P1 |
| 0  | 8  | 21 | 36 | 56 | 81 |

Detailed Execution Log:
P4: time 0 -> 8 ms, burst 8 -> 0
P2: time 8 -> 21 ms, burst 13 -> 0
P5: time 21 -> 36 ms, burst 15 -> 0
P3: time 36 -> 56 ms, burst 20 -> 0
P1: time 56 -> 81 ms, burst 25 -> 0
-----
PROCESS TABLE
-----


| PID | Burst | Start | Finish | Waiting | Turnaround |
|-----|-------|-------|--------|---------|------------|
| 1   | 25    | 56    | 81     | 56      | 81         |
| 2   | 13    | 8     | 21     | 8       | 21         |
| 3   | 20    | 36    | 56     | 36      | 56         |
| 4   | 8     | 0     | 8      | 0       | 8          |
| 5   | 15    | 21    | 36     | 21      | 36         |


-----
PERFORMANCE METRICS
-----
Average Waiting Time : 24.20 ms
Average Turnaround Time : 40.40 ms

[Main] Simulation completed. All threads terminated.
BUILD SUCCESSFUL (total time: 7 seconds)
```

Screenshot 2: Round Robin Output: Shows the Gantt chart for Round Robin scheduling with quantum q=5ms. All processes rotate in arrival order with each getting 5ms time slices. Average waiting time: 48.20ms.

run:

CSC 227: Multithreaded CPU Scheduling Simulator

Select a CPU Scheduling Algorithm:

1. Shortest Job First (SJF)
2. Round Robin (RR, q = 5ms)
3. Priority Scheduling (Non-Preemptive) with Aging

Enter your choice (1-3): 2

[Thread 2] Started: Memory Manager. Total memory = 2048MB

[Scheduler] Starting Round Robin Scheduling (q = 5ms)...

[Thread 1] Started: Reading jobs from job.txt

[Thread 2] P1 admitted to Ready Queue. Available = 1548MB

[Thread 1] Added to Job Queue: P1 [burst=25, remaining=25, priority=4, memory=500MB, state=new]

[Thread 2] P2 admitted to Ready Queue. Available = 848MB

[Thread 1] Added to Job Queue: P2 [burst=13, remaining=13, priority=3, memory=700MB, state=new]

[Scheduler] Received: P1 [burst=25, remaining=25, priority=4, memory=500MB, state=ready]

[Thread 2] P3 admitted to Ready Queue. Available = 748MB

[Scheduler] Received: P2 [burst=13, remaining=13, priority=3, memory=700MB, state=ready]

[Thread 1] Added to Job Queue: P3 [burst=20, remaining=20, priority=3, memory=100MB, state=new]

[Thread 2] P4 admitted to Ready Queue. Available = 448MB

[Thread 1] Added to Job Queue: P4 [burst=8, remaining=8, priority=1, memory=300MB, state=new]

[Scheduler] Received: P3 [burst=20, remaining=20, priority=3, memory=100MB, state=ready]

[Thread 1] Added to Job Queue: P5 [burst=15, remaining=15, priority=2, memory=400MB, state=new]

[Thread 1] Finished: All jobs inserted into Job Queue.

[Scheduler] Received: P4 [burst=8, remaining=8, priority=1, memory=300MB, state=ready]

[Thread 2] P5 admitted to Ready Queue. Available = 48MB

[Thread 2] No more jobs from Thread 1.

[Scheduler] Received: P5 [burst=15, remaining=15, priority=2, memory=400MB, state=ready]

[MEM-FREE] P4 completed. Freed 300MB. Available = 348MB

[MEM-FREE] P2 completed. Freed 700MB. Available = 1048MB

[MEM-FREE] P5 completed. Freed 400MB. Available = 1448MB

[MEM-FREE] P3 completed. Freed 100MB. Available = 1548MB

[MEM-FREE] P1 completed. Freed 500MB. Available = 2048MB

GANTT CHART

[Thread 2] Finished: All processes admitted and completed.

| P1 | P2 | P3 | P4 | P5 | P1 | P2 | P3 | P4 | P5 | P1 | P2 | P3 | P5 | P1 | P3 | P1 |
0 5 10 15 20 25 30 35 40 43 48 53 56 61 66 71 76 81

Detailed Execution Log:

P1: time 0 -> 5 ms, burst 25 -> 20
P2: time 5 -> 10 ms, burst 13 -> 8
P3: time 10 -> 15 ms, burst 20 -> 15
P4: time 15 -> 20 ms, burst 8 -> 3
P5: time 20 -> 25 ms, burst 15 -> 10
P1: time 25 -> 30 ms, burst 20 -> 15
P2: time 30 -> 35 ms, burst 8 -> 3
P3: time 35 -> 40 ms, burst 15 -> 10
P4: time 40 -> 43 ms, burst 3 -> 0
P5: time 43 -> 48 ms, burst 10 -> 5
P1: time 48 -> 53 ms, burst 15 -> 10
P2: time 53 -> 56 ms, burst 3 -> 0
P3: time 56 -> 61 ms, burst 10 -> 5
P5: time 61 -> 66 ms, burst 5 -> 0
P1: time 66 -> 71 ms, burst 10 -> 5
P3: time 71 -> 76 ms, burst 5 -> 0
P1: time 76 -> 81 ms, burst 5 -> 0

PROCESS TABLE

PID	Burst	Start	Finish	Waiting	Turnaround
1	25	0	81	56	81
2	13	5	56	43	56
3	20	10	76	56	76
4	8	15	43	35	43
5	15	20	66	51	66

PERFORMANCE METRICS

Average Waiting Time : 48.20 ms
Average Turnaround Time : 64.40 ms

[Main] Simulation completed. All threads terminated.

BUILD SUCCESSFUL (total time: 3 seconds)

Screenshot 3: Priority Scheduling Output: Shows the Gantt chart, process table with Priority and Starved columns, and the Starvation Report. Processes P1, P3, and P5 experienced starvation and received aging boosts. Average waiting time: 26.20ms.

```
run:
-----
CSC 227: Multithreaded CPU Scheduling Simulator
-----

Select a CPU Scheduling Algorithm:
1. Shortest Job First (SJF)
2. Round Robin (RR, q = 5ms)
3. Priority Scheduling (Non-Preemptive) with Aging
Enter your choice (1-3): 3
[Thread 2] Started: Memory Manager. Total memory = 2048MB

[Scheduler] Starting Priority Scheduling with Aging...
[Thread 1] Started: Reading jobs from job.txt
[Thread 2] P1 admitted to Ready Queue. Available = 1548MB
[Thread 1] Added to Job Queue: P1 [burst=25, remaining=25, priority=4, memory=500MB, state=new]
[Thread 2] P2 admitted to Ready Queue. Available = 848MB
[Thread 1] Added to Job Queue: P2 [burst=13, remaining=13, priority=3, memory=700MB, state=new]
[Thread 2] P3 admitted to Ready Queue. Available = 748MB
[Thread 1] Added to Job Queue: P3 [burst=20, remaining=20, priority=3, memory=100MB, state=new]
[Scheduler] Received: P1 [burst=25, remaining=25, priority=4, memory=500MB, state=ready]
[Thread 1] Added to Job Queue: P4 [burst=8, remaining=8, priority=1, memory=300MB, state=new]
[Scheduler] Received: P2 [burst=13, remaining=13, priority=3, memory=700MB, state=ready]
[Thread 2] P4 admitted to Ready Queue. Available = 448MB
[Scheduler] Received: P3 [burst=20, remaining=20, priority=3, memory=100MB, state=ready]
[Thread 1] Added to Job Queue: P5 [burst=15, remaining=15, priority=2, memory=400MB, state=new]
[Thread 1] Finished: All jobs inserted into Job Queue.
[Thread 2] P5 admitted to Ready Queue. Available = 48MB
[Thread 2] No more jobs from Thread 1.
[Scheduler] Received: P4 [burst=8, remaining=8, priority=1, memory=300MB, state=ready]
[Scheduler] Received: P5 [burst=15, remaining=15, priority=2, memory=400MB, state=ready]
[MEM-FREE] P2 completed. Freed 700MB. Available = 748MB
[Starvation] P1 waited 16ms > threshold 15ms
[Starvation] P3 waited 16ms > threshold 15ms
[Starvation] P5 waited 16ms > threshold 15ms
[Aging] P1 priority improved to 3
[Aging] P3 priority improved to 2
[Aging] P5 priority improved to 1
[MEM-FREE] P4 completed. Freed 300MB. Available = 1048MB
[Aging] P1 priority improved to 2
[Aging] P3 priority improved to 1
[Aging] P1 priority improved to 1
[MEM-FREE] P5 completed. Freed 400MB. Available = 1448MB
[MEM-FREE] P1 completed. Freed 500MB. Available = 1948MB
[MEM-FREE] P3 completed. Freed 100MB. Available = 2048MB
```

```

-----
[Thread 2] Finished: All processes admitted and completed.
GANTT CHART
-----
| P2 | P4 | P5 | P1 | P3 |
| 0  | 13 | 21 | 36 | 61 | 81 |

Detailed Execution Log:
P2: time 0 -> 13 ms, burst 13 -> 0
P4: time 13 -> 21 ms, burst 8 -> 0
P5: time 21 -> 36 ms, burst 15 -> 0
P1: time 36 -> 61 ms, burst 25 -> 0
P3: time 61 -> 81 ms, burst 20 -> 0

-----
PROCESS TABLE
-----
PID  Burst  Priority  Start  Finish  Waiting  Turnaround  Starved
1    25    1        36     61     36      61         Yes
2    13    3        0      13     0       13         No
3    20    1        61     81     61      81         Yes
4    8     1        13     21     13      21         No
5    15    1        21     36     21      36         Yes

-----
PERFORMANCE METRICS
-----
Average Waiting Time : 26.20 ms
Average Turnaround Time : 42.40 ms

-----
STARVATION REPORT
-----
Processes that suffered starvation and received aging:
P1
P3
P5

[Main] Simulation completed. All threads terminated.
BUILD SUCCESSFUL (total time: 3 seconds)

```

g. Additional Functionalities

Our implementation includes several features beyond the core requirements:

- **SharedQueues class:** Instead of using Java's built-in BlockingQueue, we designed a custom SharedQueues class using plain LinkedList with synchronized methods. This gives us full control over synchronization and makes the thread communication logic transparent and educational.
- **Input validation in JobLoader:** Thread 1 validates each line of job.txt, checking that priority is within range 1-30, burst time is positive, and memory is positive. Invalid lines are skipped with an error message instead of crashing the program.
- **originalPriority field in PCB:** The PCB stores the original priority separately from the current priority. This allows the process table to show how much aging has been applied to each process.
- **Starved column in Priority table:** The process table for Priority scheduling includes an extra Starved column showing Yes or No for each process, making starvation detection results immediately visible.
- **Graceful input handling:** The algorithm selection menu rejects invalid inputs (both non-numeric and out-of-range numbers) with clear error messages, looping until valid input is provided.
- **Real-time thread logging and detailed execution log:** The program prints live messages from each thread throughout execution using tags like [Thread 1] and [Scheduler]. These messages are not required but were added to prove that threads are genuinely running concurrently and to make starvation detection and aging events traceable in real time without a debugger. Additionally, after the Gantt chart we print a detailed execution log showing the exact time range and burst values for each process slice making it easy to verify the correctness of the simulation step by step.

